كلية العلوم التطبيقية - ورزازات
ⵜⴰⵖⵍⵓⵍⵜ ⵏ ⵜⵜⴻⵛⵛⵉⵔⵉⵏ ⵜⵉⵎⵙⵙⵓⵖⵜⵉⵏ - ⵡⵓⵔⵣⴰⵣⴰⵜ
FACULTE DES SCIENCES APPLIQUEES - OUARZAZATE

TRAVAUX PRATIQUES – 1

Catalogue de produits

TP autonome : logique métier

Pr. OMARI Kamal
Faculté des Sciences Appliquées – Ouarzazate
IAIL S5 | Année universitaire 2026–2027

Réalisé par :
Younes Ait Elkaid

Sommaire

1	Étape 1 – Préparer l'espace de travail	3
2	Étape 2 – Créer le tableau des produits	4
3	Étape 3 – Afficher le catalogue	5
4	Étape 4 – Trouver les produits disponibles	6
5	Étape 5 – Calculer la valeur du stock	7
6	Étape 6 – Trouver le produit le moins cher	8
7	Étape 7 – Construire le résumé	9
8	Étape 8 – Cas de test imposés	10
9	Étape 9 – Défi : la déstructuration	11
10	Étape 10 – Défi : écran mobile avec Expo	12
11	Code final complet : produits.js	13

ÉTAPE 1

Préparer l'espace de travail

CODE (COMMANDES DU TERMINAL)

```
node --version
mkdir tp2-catalogue
cd tp2-catalogue
```

EXÉCUTION DANS LE TERMINAL

```
● ● ●  Terminal — tp2-catalogue

$ node --version
v22.22.2
$ mkdir tp2-catalogue
$ cd tp2-catalogue
```

RÉSULTAT :

> Le terminal affiche la version de Node.js installée : **`v22.22.2`**.
>
> Les commandes **`mkdir`** et **`cd`** ne produisent aucun message : le dossier **`tp2-catalogue`** est créé et ouvert.

EXPLICATION :

`node --version` vérifie que Node.js est installé. **`mkdir tp2-catalogue`** crée le dossier demandé par le sujet et **`cd tp2-catalogue`** permet de s'y placer. Le fichier **`produits.js`** sera ensuite créé dans ce dossier avec Visual Studio Code.

ÉTAPE 2

Créer le tableau des produits

CODE (FICHIER PRODUITS.JS)

```
const produits = [
  { id: 1, nom: 'Clavier', categorie: 'Accessoires', prix: 120, stock: 5 },
  { id: 2, nom: 'Souris', categorie: 'Accessoires', prix: 80, stock: 8 },
  { id: 3, nom: 'Écran', categorie: 'Informatique', prix: 900, stock: 0 },
  { id: 4, nom: 'Casque', categorie: 'Accessoires', prix: 240, stock: 3 }
];
console.log('Nombre de produits :', produits.length);
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
```

RÉSULTAT :

```
Nombre de produits : 4
```

EXPLICATION :

`produits` est un tableau de quatre objets. Chaque objet possède les propriétés **`id`**, **`nom`**, **`categorie`**, **`prix`** et **`stock`**, avec des identifiants numériques distincts (1 à 4). **`const`** empêche de réaffecter le tableau, et la propriété **`length`** donne le nombre de produits.

ÉTAPE 3

Afficher le catalogue

CODE (À AJOUTER À PRODUITS.JS)

```
for (const p of produits) {
  console.log(`${p.nom} : ${p.prix} MAD`);
}
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue
```

```
$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
```

RÉSULTAT :

Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD

EXPLICATION :

La boucle **`for...of`** parcourt le tableau produit par produit. À chaque tour, **`p`** représente un objet et le gabarit entre accents graves affiche son nom et son prix. Les quatre produits apparaissent, y compris l'écran dont le stock est nul.

ÉTAPE 4

Trouver les produits disponibles

CODE (À AJOUTER À PRODUITS.JS)

```
const disponibles = produits.filter((p) => p.stock > 0);
console.log('Produits disponibles :', disponibles.length);
```

EXÉCUTION DANS LE TERMINAL

```
● ● ●  Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
```

RÉSULTAT :

```
Produits disponibles : 3
```

EXPLICATION :

`filter()` crée un nouveau tableau contenant uniquement les produits qui vérifient la condition **`stock > 0`**. Le tableau de départ n'est pas modifié. L'écran (stock 0) est écarté, il reste donc 3 produits disponibles.

ÉTAPE 5

Calculer la valeur du stock

CODE (À AJOUTER À PRODUITS.JS)

```
function valeurStock(liste) {
  return liste.reduce((total, p) => total + p.prix * p.stock, 0);
}
console.log('Valeur du stock :', valeurStock(produits), 'MAD');
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
```

RÉSULTAT :

`Valeur du stock : 1960 MAD`

EXPLICATION :

La fonction **`valeurStock`** utilise **`reduce()`** : l'accumulateur démarre à **`0`** et reçoit **`prix × stock`** pour chaque produit. Le calcul est 120×5 + 80×8 + 900×0 + 240×3 = 600 + 640 + 0 + 720 = 1960 MAD. Avec une liste vide, **`reduce()`** renvoie directement la valeur initiale **`0`**.

ÉTAPE 6

Trouver le produit le moins cher

CODE (À AJOUTER À PRODUITS.JS)

```
function moinsCher(liste) {
  if (liste.length === 0) return null;
  return liste.reduce((min, p) => (p.prix < min.prix ? p : min));
}
const pasCher = moinsCher(disponibles);
if (pasCher === null) {
  console.log('Aucun produit disponible');
} else {
  console.log('Moins cher :', pasCher.nom, '-', pasCher.prix, 'MAD');
}
```

EXÉCUTION DANS LE TERMINAL

```
● ● ●  Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
Moins cher : Souris - 80 MAD
```

RÉSULTAT :

```
Moins cher : Souris - 80 MAD
```

EXPLICATION :

La fonction **`moinsCher`** renvoie **`null`** si la liste est vide, afin d'afficher le message « Aucun produit disponible » au lieu de provoquer une erreur. Sinon, **`reduce()`** garde à chaque comparaison le produit dont le prix est le plus faible. Parmi les produits disponibles, c'est la Souris à 80 MAD.

ÉTAPE 7

Construire le résumé

CODE (À AJOUTER À PRODUITS.JS)

```
const resume = produits.map(
  (p) => `${p.nom} - ${p.prix} MAD - ${p.stock} unités`
);
for (const ligne of resume) {
  console.log(ligne);
}
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue
```

```
$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
Moins cher : Souris - 80 MAD
Clavier - 120 MAD - 5 unités
Souris - 80 MAD - 8 unités
Écran - 900 MAD - 0 unités
Casque - 240 MAD - 3 unités
```

RÉSULTAT :

Clavier - 120 MAD - 5 unités
Souris - 80 MAD - 8 unités
Écran - 900 MAD - 0 unités
Casque - 240 MAD - 3 unités

EXPLICATION :

`map()` transforme chaque produit en une ligne de texte au format **`Nom - prix MAD - stock unités`** et renvoie un nouveau tableau de chaînes. La boucle **`for...of`** affiche ensuite ces lignes.

ÉTAPE 8

Cas de test imposés

CODE (À AJOUTER À PRODUITS.JS)

```
// Test 1 : l'écran à stock nul n'est pas dans la liste disponible
console.log('Écran disponible ?', disponibles.some((p) => p.nom === 'Écran'));
// Test 2 : tous les stocks à zéro (copie, sans modifier les objets de départ)
const stocksZero = produits.map((p) => ({ ...p, stock: 0 }));
const dispoZero = stocksZero.filter((p) => p.stock > 0);
const minZero = moinsCher(dispoZero);
console.log('Disponibles (stocks à 0) :', dispoZero.length);
console.log(minZero === null ? 'Aucun produit disponible' : minZero.nom);
// Test 3 : catalogue vide
console.log('Valeur (catalogue vide) :', valeurStock([]), 'MAD');
// Test 4 : vérification des données
function estValide(produit) {
  return Number.isFinite(produit.prix) && Number.isFinite(produit.stock) &&
         produit.prix >= 0 && produit.stock >= 0;
}
console.log('Données valides ?', produits.every(estValide));
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
Moins cher : Souris - 80 MAD
Clavier - 120 MAD - 5 unités
Souris - 80 MAD - 8 unités
Écran - 900 MAD - 0 unités
Casque - 240 MAD - 3 unités
Écran disponible ? false
Disponibles (stocks à 0) : 0
Aucun produit disponible
Valeur (catalogue vide) : 0 MAD
Données valides ? true
```

RÉSULTAT :

Écran à stock nul : **`Écran disponible ? false`**.

Tous les stocks à zéro : **`Disponibles : 0`** puis **`Aucun produit disponible`**, sans plantage.

Catalogue vide : **`Valeur (catalogue vide) : 0 MAD`**.

Vérification des données : **`Données valides ? true`**.

EXPLICATION :

Les quatre tests sont réussis. Pour les stocks à zéro, **`{ ...p, stock: 0 }`** crée des copies : les objets de départ ne sont pas modifiés. **`estValide`** vérifie que prix et stock sont des nombres finis non négatifs, et **`every()`** contrôle tous les produits.

ÉTAPE 9

Défi : la déstructuration

CODE (À AJOUTER À PRODUITS.JS)

```
const { nom, prix } = pasCher;
console.log(`Le moins cher : ${nom} (${prix} MAD)`);
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ node produits.js
Nombre de produits : 4
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
Moins cher : Souris - 80 MAD
Clavier - 120 MAD - 5 unités
Souris - 80 MAD - 8 unités
Écran - 900 MAD - 0 unités
Casque - 240 MAD - 3 unités
Écran disponible ? false
Disponibles (stocks à 0) : 0
Aucun produit disponible
Valeur (catalogue vide) : 0 MAD
Données valides ? true
Le moins cher : Souris (80 MAD)
```

RÉSULTAT :

```
Le moins cher : Souris (80 MAD)
```

EXPLICATION :

La déstructuration **`const { nom, prix } = pasCher`** extrait en une seule instruction les propriétés **`nom`** et **`prix`** du produit disponible le moins cher, puis elles sont utilisées directement dans le message.

ÉTAPE 10

Défi : écran mobile avec Expo

CODE (NOUVEAU CONTENU DE APP.TSX)

```
import { View, Text, StyleSheet } from 'react-native';

const produits = [
  { id: 1, nom: 'Clavier', categorie: 'Accessoires', prix: 120, stock: 5 },
  { id: 2, nom: 'Souris', categorie: 'Accessoires', prix: 80, stock: 8 },
  { id: 3, nom: 'Écran', categorie: 'Informatique', prix: 900, stock: 0 },
  { id: 4, nom: 'Casque', categorie: 'Accessoires', prix: 240, stock: 3 }
];
const disponibles = produits.filter((p) => p.stock > 0);
const valeur = produits.reduce((t, p) => t + p.prix * p.stock, 0);

export default function App() {
  return (
    <View style={styles.ecran}>
      <Text style={styles.titre}>Catalogue de produits</Text>
      <Text style={styles.info}>Produits disponibles : {disponibles.length}</Text>
      <Text style={styles.info}>Valeur du stock : {valeur} MAD</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  ecran: { flex: 1, justifyContent: 'center', padding: 26,
           backgroundColor: '#F2F7FD' },
  titre: { fontSize: 24, fontWeight: 'bold', color: '#00235C',
           marginBottom: 16 },
  info: { fontSize: 18, paddingVertical: 6, color: '#003796' }
});
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ npx expo start
# Scanner le QR code affiché avec l'application Expo Go
```

RÉSULTAT :

> Un écran affiche le titre « Catalogue de produits », **`Produits disponibles : 3`** et **`Valeur du stock : 1960 MAD`**.

EXPLICATION :

L'écran utilise seulement les composants **`View`** et **`Text`**, sans outil de navigation. Les valeurs sont calculées avec **`filter()`** et **`reduce()`**, exactement comme dans la console, puis insérées dans le JSX entre accolades.

CODE FINAL

Code final complet : produits.js

CODE (FICHIER PRODUITS.JS)

```
const produits = [
  { id: 1, nom: 'Clavier', categorie: 'Accessoires', prix: 120, stock: 5 },
  { id: 2, nom: 'Souris', categorie: 'Accessoires', prix: 80, stock: 8 },
  { id: 3, nom: 'Écran', categorie: 'Informatique', prix: 900, stock: 0 },
  { id: 4, nom: 'Casque', categorie: 'Accessoires', prix: 240, stock: 3 }
];
function estValide(produit) {
  return Number.isFinite(produit.prix) && Number.isFinite(produit.stock) &&
         produit.prix >= 0 && produit.stock >= 0;
}
function valeurStock(liste) {
  return liste.reduce((total, p) => total + p.prix * p.stock, 0);
}
function moinsCher(liste) {
  if (liste.length === 0) return null;
  return liste.reduce((min, p) => (p.prix < min.prix ? p : min));
}
const disponibles = produits.filter((p) => p.stock > 0);
const resume = produits.map(
  (p) => `${p.nom} - ${p.prix} MAD - ${p.stock} unités`
);
const pasCher = moinsCher(disponibles);
console.log('Données valides ?', produits.every(estValide));
for (const p of produits) console.log(`${p.nom} : ${p.prix} MAD`);
console.log('Produits disponibles :', disponibles.length);
console.log('Valeur du stock :', valeurStock(produits), 'MAD');
if (pasCher === null) {
  console.log('Aucun produit disponible');
} else {
  const { nom, prix } = pasCher;
  console.log(`Le moins cher : ${nom} (${prix} MAD)`);
}
for (const ligne of resume) console.log(ligne);
```

EXÉCUTION DANS LE TERMINAL

```
Terminal — tp2-catalogue

$ node produits.js
Données valides ? true
Clavier : 120 MAD
Souris : 80 MAD
Écran : 900 MAD
Casque : 240 MAD
Produits disponibles : 3
Valeur du stock : 1960 MAD
Le moins cher : Souris (80 MAD)
Clavier - 120 MAD - 5 unités
Souris - 80 MAD - 8 unités
Écran - 900 MAD - 0 unités
Casque - 240 MAD - 3 unités
```

RÉSULTAT :

Produits disponibles : 3 | Valeur du stock : 1960 MAD | Souris (80 MAD)

EXPLICATION :

Le fichier final regroupe les données, les fonctions et les opérations **`filter()`**, **`map()`**, **`reduce()`**. Les résultats correspondent à ceux du sujet : 3 produits disponibles, 1960 MAD de stock, la Souris la moins chère.